OSS-Next — Blockers Report

2026-07-24 (updated) · Branch `integrate-wp-render`
What is blocked, who can unblock it, and exactly what is needed.

> **One backend fix unblocks a working purchase: the Django checkout crash (B1).** Both front-end payment defects are closed and verified against the real Braintree sandbox, and credentials are configured. B1 is the last thing between the current build and an end-to-end order — three further items sit behind it.

> **Resolved since the earlier 2026-07-24 report** — client answers:
>
> - **Order confirmation emails** — confirmed **Django owns them**. No email code expected on the Next.js side. Closed.
> - **Address book** — confirmed **edit-address stays singular** (one billing + one shipping). Multi-address isn't wanted; not a gap. Closed.
> - **Newsletter (registered users)** — built `/my-account/newsletter` with subscribe / unsubscribe in the account nav. State is authoritative: it reads the profile's `is_subscribed` flag (which our normalizer had been dropping) and writes it back after a toggle. The public homepage widget is deferred to the new homepage design.

Owner	Open items	Nature
Backend team	3	Code fixes and a payment-status verification
Client	3	Business/product decisions; no engineering blocked behind them
Frontend	0	Nothing actionable outstanding

Backend Blockers

B1 · CRITICAL Checkout endpoint crashes on every call

`POST /api/orders/checkout` returns 500 on every request:

```
ValueError: Currency formatting is not possible using the 'C' locale.
app/orders/views.py, line 89 — locale.currency(float(item.total), grouping=True)
```

The server has no locale configured. **The order is saved before the crash** — real orders were confirmed created despite every attempt 500ing. With payment working, a customer would be charged, have an order created, and still see a failure.

> **What we need:** `locale.setlocale(locale.LC_ALL, 'en_US.UTF-8')` at startup, or drop `locale.currency()` and format directly (e.g. `f"${amount:,.2f}"`) — the more robust option, independent of installed locales.

B2 Order status never becomes `paid`; verify the transaction

The checkout payload now sends `status: "paid"` (the order is only created after a successful charge) plus `payment_method` and the Braintree `transaction_id`. The backend should honor that status — **but must confirm the `transaction_id` against Braintree before trusting it.** The order POST is browser-made, so a blindly trusted status would let a forged request mint a free paid order (same class as the client-amount hole already fixed on our side).

> **What we need:** on order-create, look up the `transaction_id` via the Braintree API, confirm it's valid and the amount matches, then set `paid`. Also confirm `transaction_id` is persisted so payments can be traced back from an order. (Note: our charge uses submit-for-settlement, so a fresh transaction reads `submitted_for_settlement` and settles hours later — treat that as paid, not unpaid.)

B3 The pages API serves malformed CSS

The CSS minifier strips `/* */` delimiters but leaves the comment text, and emits unbalanced braces:

```
once slick adds its class,show it smoothly .shipping-container-slider…{…}
}.slick-slide:not(.slick-active){opacity:0!important}
```

Strict parsers reject it; **browsers discard every rule after the fault**, so the live WordPress site loses styling too. We work around it with a tolerant parser, but the source is wrong.

> **What we need:** fix the minifier so comments are removed whole and braces stay balanced.

Client Decisions

> None block engineering — they block *starting* it. C1 and C2 carry active customer-facing risk, and both have a cheap interim mitigation that needs no product decision.

C1 · CUSTOMER-FACING RISK The homepage quote form silently discards submissions

The form shows **"✓ Quote Request Sent — We'll be in touch!"** while the customer's name, email, phone, ZIP and container details go **nowhere**. Every lead is lost, and the visitor believes they've contacted the business.

> **What we need:** the Zoho Forms account and form details to wire the real destination. **Until then, disable the form or remove the success message** — the current behaviour is worse than no form.

C2 · CUSTOMER-FACING RISK The site advertises live chat that doesn't exist

The homepage trust strip promises "Phone, Email & Chat" support. There's no chat feature, and none planned.

> **What we need:** a decision to build chat, or approval to change the copy. **The copy fix takes minutes and needs no engineering decision.**

C3 Shipment tracking in order history

Order history shows real status, but there's no carrier or tracking number in the data, so customers can't track a delivery.

> **What we need:** confirmation that tracking is wanted, and whether the backend can return a carrier + tracking number on the order record.

Deferred, Not Blocked

Item	Status
Newsletter — public homepage widget	Waiting on the new homepage design that will drive it. The registered-user page is already built.
End-to-end purchase verification	Behind B1 — front end built and verified, endpoint 500s.
Populated order-history rendering	Behind B1 — no order can be created to test against.

Generated 2026-07-24 from `docs/audits/AUDIT_REQUIREMENTS.md`, `API_INTEGRATION_STATUS.md` and `API_TRIGGER_CHECKLIST.md`. The `.md` files are the living versions. Supersedes the earlier `BLOCKERS_2026-07-23.pdf` and the first same-day cut. All backend findings were reproduced against the live backend, not inferred from code.